

Linux Learning Module

LINUX LEARNING MODULE

Prepared as a self-learning textbook for Linux systems architecture,
administration, and high-performance computing environments.

Created by: Ankit Kumar

Contents

PART 1 — LINUX & UNIX FUNDAMENTALS	1
PART 2 — LINUX SYSTEM ARCHITECTURE & THE KERNEL	3
PART 3 — COMMAND LINE INTERFACE (CLI) & TERMINAL OPERATIONS	5
PART 4 — LINUX DIRECTORY HIERARCHY & STORAGE MANAGEMENT	7
PART 5 — FILE & DIRECTORY MANIPULATION	14
PART 6 — TEXT DISPLAY, INSPECTION, AND PATTERN SEARCHING	18
PART 7 — INPUT/OUTPUT STREAMS, REDIRECTION, & COMMAND PIPELINES	22
PART 8 — MULTI-USER SECURITY, PERMISSIONS, & ACCESS CONTROL	24
PART 9 — PROCESS MANAGEMENT, CPU SCHEDULING, & SYNCHRONIZATION	27
PART 10 — OPERATING SYSTEM MEMORY MANAGEMENT	32
PART 11 — LINUX PACKAGE MANAGEMENT SYSTEMS	34
PART 12 — THE VIM TEXT EDITOR	37
PART 13 — SHELL SCRIPTING, CONTROL STRUCTURES, & AUTOMATION	38
PART 14 — SHELL VARIABLES, ENVIRONMENT CONFIGURATION, & HPC MOD- ULES	42
PART 15 — SECURE SHELL (SSH) & REMOTE DATA TRANSFER	44
COMPREHENSIVE PRACTICAL LAB MANUAL	46
FINAL SYSTEM REFERENCE & HANDBOOK COMPENDIUM	58

PART 1 — LINUX & UNIX FUNDAMENTALS

1.1 Historical Evolution: Unix to Linux

1. Definition

The **Unix** operating system is a portable, multi-tasking, multi-user, multi-processing operating system developed in 1969 at AT&T's Bell Laboratories by Dennis Ritchie, Ken Thompson, and colleagues, originally written in the C programming language as an architectural successor to Multics. **Linux** is a Unix-like operating system built upon an independent monolithic kernel originally conceived and developed by Linus Torvalds in 1991 at the University of Helsinki.

```
+-----+
| HISTORICAL TIMELINE OVERVIEW |
| |
| 1969 1985 1990 1991 1992 1993 1994 |
| | | | | | | |
| Unix Minix POSIX Linux Ext1/SLS Debian, Linux |
| (Bell Labs) (Tanenbaum)(Standard)(Torvalds) Released Slackware Kernel |
| Red Hat 1.0 |
| |
| 1996 2001 2003 2007 2008 2020s |
| | | | | | | |
| GNOME Ubuntu, Kernel Android Kernel/ 100% Top500 |
| Desktop Ext3 2.6 Mobile Ext4 Supercomputers |
+-----+
```

2. Theory

- **The Unix Heritage:** Unix emerged to solve rigid operating limitations of early batch systems. Rewriting it in C rendered the operating system portable across disparate hardware platforms, decoupling software execution from machine architecture.
- **The POSIX Standard (1990):** The Portable Operating System Interface (POSIX) was established by the IEEE to standardize the application programming interface (API), shell specifications, and utility interfaces across Unix variants. Linux adheres strictly to POSIX guidelines, allowing compliant Unix code to compile cleanly on Linux systems.
- **Open Source Licensing:**
 - **GNU General Public License (GPL):** A copyleft license requiring that any distributed copy or derivative work must provide the underlying source code under the exact same licensing terms.

Berkeley Software Distribution (BSD): A permissive license permitting unrestricted proprietary derivative works without mandating the distribution of modified source code.

- **Linux in Supercomputing (HPC):** Linux holds 100% of the TOP500 Supercomputer operating system market share.

TOP500 SUPERCOMPUTER OPERATING SYSTEM DISTRIBUTION

Total Systems Tracked: 500 / Linux Share: 100%

Operating System Distribution	Share (%)
Linux (Generic / Custom Enterprise)	46.8%
CentOS	19.2%
Cray Linux Environment	15.4%
HPE Cray OS	~5.0%
Red Hat Enterprise Linux (RHEL)	~4.2%
bullx SCS	~3.1%
Ubuntu (20.04 LTS / Server)	~2.5%
SUSE Linux Enterprise (SLES)	~1.8%
CentOS Linux 7	~1.2%
Others (Scientific Linux, etc.)	~0.8%

3. Explain in Simple Language

Imagine Unix as a blueprint for high-performance sports cars designed by elite automotive engineers in 1969. For decades, major corporations built their own expensive, proprietary versions. In 1991, Linus Torvalds designed an open-access sports engine called the Linux kernel. Combined with free tools (GNU), anyone could build, tune, or inspect the engine without paying licensing fees. Because thousands of engineers continually optimize it, this engine powers both smartphones and the world's most powerful supercomputers.

4. Why Is It Used?

Linux provides unmatched stability, granular security controls, transparent open-source codebases, zero vendor licensing lock-in, and scalability across architectures from low-power ARM microcontrollers to multi-petabyte supercomputing clusters.

5. Comparison: Linux vs. Proprietary Unix

Architectural Dimension	Linux	Proprietary Unix (Solaris, AIX, HP-UX)
Licensing Model	Open Source (GNU GPL)	Commercial / Proprietary
Source Code Access	Fully accessible and auditable	Closed Source
Kernel Type	Monolithic (with dynamically loadable modules)	Monolithic, Microkernel, or Hybrid (OS specific)
Filesystems	Ext2, Ext3, Ext4, XFS, Btrfs	UFS, ZFS, JFS, GPFS, VxFS
Hardware Platform	Universal (x86_64, ARM, RISC-V, POWER)	Tied to vendor hardware (SPARC, POWER, PA-RISC)
HPC Presence	100% share of TOP500 supercomputing systems	Historical only; displaced by Linux

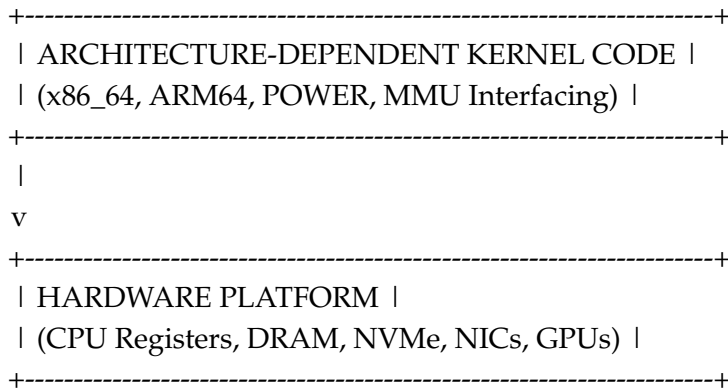
PART 2 — LINUX SYSTEM ARCHITECTURE & THE KERNEL

2.1 Layered Architecture Overview

```

+-----+
| USER APPLICATIONS |
| (Shells, Compilers, Daemons, Text Editors, Web Browsers) |
+-----+
|
v
+-----+
| GNU C LIBRARY (glibc) |
| Standard API Wrapper Functions & POSIX |
+-----+
|
===== USER SPACE / KERNEL SPACE =====
| (Hardware Ring Transition)
v
+-----+
| SYSTEM CALL INTERFACE |
| (sys_read, sys_write, sys_fork, sys_execve) |
+-----+
| THE KERNEL |
| +-----+ |
| | Process Scheduler | Memory Manager | Virtual File System | |
| | (CFS, Real-time) | (Paging/Virtual) | (VFS Abstraction) | |
| +-----+ +-----+ +-----+ |
| | Network Stack & Device Drivers | |
| +-----+ |

```



1. Definition

The **Linux Architecture** is a multi-tiered, protected operating system model structured into **User Space** (unprivileged execution) and **Kernel Space** (privileged ring-0 execution), mediated strictly by the **System Call Interface** and managed by the **Kernel** to abstract hardware components.

2. Theory

- **User Space:** A protected memory boundary where user programs, shells, development runtimes, and user applications run. Memory isolation prevents a faulty user process from corrupting another process or crashing the underlying kernel.
- **Kernel Space:** The high-privilege memory domain where the operating system core resides. The kernel has complete, direct access to physical memory addresses, hardware controllers, and CPU execution rings.
- **GNU C Library (glibc):** Acts as the standard runtime interface between user-space programs and the operating system kernel. It converts application programming calls into system calls.
- **System Call Interface:** A secure software gate. When a process requires low-level operations (e.g., `sys_read` to read a disk or `sys_fork` to create a process), execution changes from user mode to kernel mode via software interrupts or hardware-accelerated instructions (`syscall` / `sysenter`).
- **The Shell:** A user-space command-language interpreter acting as the primary interface between the user and the system's kernel, converting human-entered commands into low-level executions.

3. Explain in Simple Language

Think of the Linux system as an upscale restaurant:

-

User Applications: Customers seated in the dining room (User Space). They cannot wander into the kitchen or operate stoves directly.

- **The Shell & glibc:** The head waiter who takes orders, formats them correctly, and carries them to the kitchen doors.
- **The System Call Interface:** The pass-through window between the dining room and kitchen.
- **The Kernel:** The executive kitchen staff operating inside Kernel Space. They handle cooking equipment, control ingredients (memory), and schedule stove time (CPU).
- **The Hardware Platform:** The raw appliances: gas lines, ovens, refrigerators, and prep tables.

PART 3 — COMMAND LINE INTERFACE (CLI) & TERMINAL OPERATIONS

3.1 Anatomy of the Terminal & Command Prompt

1. Definition

The **Command Line Interface (CLI)** is a text-based environment enabling users and administrators to execute commands, automate administrative workloads, and manage processes by entering structured text input evaluated by an underlying shell interpreter.

2. Anatomy of the Prompt

The standard interactive Bash command prompt displays essential contextual system metadata:

```
[ karuna @ login01 ~ ] $
| | | |
| | | +--> Privilege Identifier ($: Non-root, #: Root)
| | +-----> Current Working Directory (~ means Home)
| +-----> System Hostname (Machine ID)
+-----> Current Authenticated Username
```

- **Username:** Identifies the executing user context, establishing initial discretionary access rights.
- **Hostname:** Displays the identity of the current machine, which is critical when administering clustered nodes or remote servers.
- **Working Directory:** Shows the current location within the filesystem tree. The tilde symbol (~) is a standard shortcut representing the active user's home directory.

- **Privilege Level:**
 - \$ denotes an unprivileged, non-root user shell session.
 - # denotes the superuser (root) administrative session, which has unrestricted system privileges.

3. Command Syntax Architecture

Every standard Linux terminal command adheres to a predictable structure:

Command [Options/Flags] [Arguments]

```
hostname -f
| |
Command Option (Long / FQDN output)
```

```
ls -l /home/karuna
| | |
Command Option (Format) Argument (Target Path)
```

- **Command:** The executable binary, shell built-in, or script invoked by the user (e.g., `hostname`, `ls`, `mkdir`).
- **Options (Flags):** Arguments that alter execution behavior. Standard conventions include:
 - Short options: Single dash followed by single letters (e.g., `-l`, `-a`, `-h`).
 - Long options: Double dashes followed by descriptive keywords (e.g., `--all`, `--human-readable`, `--help`).
- **Arguments:** The target data, filenames, or directory paths the command operates on.

4. Standard Console Control Shortcuts

```
+-----+
| Shortcut | Operational Action & Kernel Signal Mapping |
+-----+
| <Enter> | Submits the command line buffer for execution. |
| <Tab> | Auto-completes commands, variables, or filesystem paths. |
| <Tab><Tab> | Emits a double tone and lists all matching possibilities. |
| <Up-Arrow> | Traverses command history backward in time. |
```

```

| <Down-Arrow> | Traverses command history forward in time. |
| <Ctrl-C> | Transmits SIGINT (Signal 2) to interrupt the foreground process. |
| <Ctrl-Z> | Transmits SIGTSTP (Signal 20) to suspend the foreground process. |
| <Ctrl-D> | Transmits EOF (End-Of-File), closing standard input. |
| <Ctrl-L> | Clears the active terminal viewport (equivalent to 'clear'). |
+-----+

```

PART 4 — LINUX DIRECTORY HIERARCHY & STORAGE MANAGEMENT

4.1 The Filesystem Hierarchy Standard (FHS)

1. Definition

The **Linux Filesystem** is a single-rooted, unified, hierarchical inverted-tree structure originating at the root directory (/), providing a standardized directory layout for operating system components, dynamic kernel metrics, hardware access points, and user data.

```

/ (Root Directory)
+-----+
| | | | |
/bin /sbin /etc /home /dev
(Essential User (Administrative (System Config (User Home (Device Nodes
Binaries) Binaries) Files) Dirs) Files)
| | | | |
/usr /proc /var /tmp /opt
(Secondary (Virtual Kernel (Variable Data, (Temporary (Optional Add-on
Hierarchy) Process Stats) Logs, Spools) Storage) Software)
| | |
/boot /mnt, /media /root
(Kernel & GRUB) (Mount Points) (Superuser Home)

```

2. Directory Functional Reference

- / (Root): The top-level ancestor of the entire filesystem tree. Every file, directory, mounted disk, and network share branches from /.
- /bin: Essential system command binaries accessible to all users (e.g., ls, cp, cat). In modern systems, this is often symlinked to /usr/bin.
- /sbin: System management binaries reserved for administrative tasks and recovery (e.g., fdisk, useradd, mkfs). Symlinked to /usr/sbin on modern setups.
- /etc: Host-specific configuration files controlling system services, networking, and user registries (e.g., /etc/passwd, /etc/fstab, /etc/hosts).

- | `/home`: Base directory containing individual non-root user workspaces. Users have write permissions inside their respective subdirectories (e.g., `/home/karuna`).
- | `/root`: The isolated home directory of the root superuser, kept off the `/home` mount to allow emergency administration if separate partitions fail to load.
- | `/dev`: Special device files mapping system hardware (e.g., disks `/dev/sda`, terminals `/dev/pts`, pseudo-devices `/dev/null`, `/dev/random`).
- | `/proc`: A pseudo-filesystem generated on the fly by the Linux kernel. It does not occupy disk space, exposing real-time kernel data structures, process mappings, and hardware parameters.
- | `/tmp`: Temporary scratch space accessible to all applications and users. Files here are typically purged on system reboot or by automated systemd timers.
- | `/var`: Variable data storage for files that grow dynamically during system operations, such as logs (`/var/log`), mail spools, and databases.
- | `/opt`: Add-on application software packages installed outside core distribution repositories.
- | `/boot`: Static storage holding the Linux kernel binary (`vmlinuz`), initial RAM disk images (`initramfs`), and the GRUB bootloader configuration.

4.2 Linux Filesystem Types & Storage Management

1. Theory: Inodes and Journaling

- | **The Inode Concept:** An *inode* (index node) is a fixed-size data structure that stores all metadata about a filesystem object except its filename and file content data blocks. Inodes track:
 - | File size, block allocations, and physical disk locations.
 - | Discretionary Access Control (DAC) permissions and ownership IDs (UID, GID).
 - | Timestamps: access time (`$atime$`), modification time (`$mtime$`), and metadata change time (`$ctime$`).
 - | The filename itself is stored inside a directory data block mapped to the corresponding inode number.
-

Journaling: A filesystem mechanism that records proposed structure changes into a dedicated on-disk circular log (the *journal*) before writing updates to main data blocks. If power fails or the system crashes, the operating system recovers consistency by replaying or discarding the journal log without needing exhaustive disk scans.

Metadata Journaling: Writes only structural changes to the log, balancing crash safety with speed.

Full Data Journaling: Records all metadata and underlying data modifications, offering maximum crash safety at the cost of disk throughput.

2. Evolution of Linux Filesystems

+-----+						
FILESYSTEM EVOLUTION MATRIX						
+-----+-----+-----+-----+-----+-----+						
Filesystem	Year	Journaling	Max File Size	Max Volume	Notes /	
	Size	Highlights				
+-----+-----+-----+-----+-----+-----+						
Ext1	1992	None	2 GB	2 GB	First Ext;	
	no symlinks					
Ext2	1993	None	2 TB	32 TiB	Remi Card;	
	simple, fast					
Ext3	2001	Yes	2 TB	16-32 TiB	Tweedie;	
	Ext2+journal					
Ext4	2008	Yes	16 TB	1 EB	Extents,	
	(4KB blocks)	(16KB blocks)	delalloc			
XFS	1994	Yes	8 EB	8 EB	SGI; 64-bit,	
	B-Trees, HPC					
+-----+-----+-----+-----+-----+-----+						

- **Ext (Extended File System - 1992):** The first dedicated Linux filesystem, introducing basic abstraction over the Minix filesystem.

- *Limitations:* Lacked journaling, lacked symbolic links, had an absolute file size ceiling of 2 GB, and lacked extended attributes.

- **Ext2 (Second Extended File System - 1993):** Designed by Rémi Card to overcome Ext limitations. Supports full Unix permissions, large volumes, directory indexing, and offline volume resizing. Still useful for flash storage or boot partitions where journaling write overhead is undesirable.

- **Ext3 (Third Extended File System - 2001):** Designed by Stephen Tweedie. Added crash-resilient journaling directly to the Ext2 metadata architecture, allowing seamless, in-place conversion from Ext2 without reformatting.

- **Ext4 (Fourth Extended File System - 2008):** Modernized the Ext lineage by replacing traditional block mapping with *extents* (contiguous ranges of physical blocks). Introduces delayed block allocation (*delalloc*), multiblock allocation, internal journal checksumming, and online defragmentation.

- **XFS (Silicon Graphics - 1994):** A 64-bit, high-throughput, enterprise journaling filesystem designed for high parallel I/O, large files, and large storage arrays. Uses B-Trees for metadata indexing, supports extents, delayed allocation, and online expansion via `xfs_growfs`. Default filesystem in Enterprise Linux (RHEL/CentOS).

4.3 Storage Provisioning Commands

COMMAND: fdisk

- **PURPOSE:** Low-level utility for viewing, creating, and modifying MBR/GPT partition tables.

- **SYNTAX:** `fdisk [options] <device_node>`

- **BREAKDOWN:**

- `fdisk` → The partition table manipulation utility.

- `-l` → Lists partition layouts and geometry on target storage devices.

- `/dev/sdb` → The target physical/virtual storage device node.

- **EXAMPLE:**

Bash

```
sudo fdisk -l /dev/sda
```

```
sudo fdisk /dev/sdb
```

- **EXPECTED OUTPUT (fdisk -l /dev/sda):**

Disk /dev/sda: 22.3 GiB, 23953276928 bytes, 46783744 sectors

Units: sectors of 1 * 512 = 512 bytes

Sector size (logical/physical): 512 bytes / 512 bytes

I/O size (minimum/optimal): 512 bytes / 512 bytes

Disklabel type: dos

Disk identifier: 0xf468ae43

```
Device Boot Start End Sectors Size Id Type
/dev/sda1 2048 2099199 2097152 1G 83 Linux
/dev/sda2 2099200 46782463 44683264 21.3G 8e Linux LVM
```

- **EXPLANATION:** Displays disk sector sizing, UUID, and the current partition layout, identifying start and end block boundaries.

- **ALTERNATIVE:** gdisk (for GPT partition schemes) or parted.

- **WHEN TO USE:** Preparing new hard drives, LUNs, or virtual disks before applying filesystems.

- **COMMON MISTAKE:** Partitioning the wrong device (e.g., partitioning /dev/sda instead of /dev/sdb), which can destroy operational OS tables.

- **SAFETY: [!] WARNING:** Committing partition writes with w inside interactive fdisk commits destructive table updates to disk.

COMMAND: mkfs.ext4

- **PURPOSE:** Builds an Ext4 filesystem structure on a specified target partition.

- **SYNTAX:** mkfs.ext4 [options] /dev/<target_partition>

- **BREAKDOWN:**

- mkfs.ext4 → Instantiates the Ext4 filesystem formatter.
- /dev/sdb1 → The raw storage partition target.

- **EXAMPLE:**

```
Bash
sudo mkfs.ext4 /dev/sdb1
```

- **EXPECTED OUTPUT:**

```
mke2fs 1.45.6 (20-Mar-2020)
Creating filesystem with 1048576 4k blocks and 262144 inodes
Filesystem UUID: b21d3f82-a74e-4b6a-93ec-e82245b08491
Superblock backups stored on blocks:
32768, 98304, 163840, 229376, 294912, 819200, 884736

Allocating group tables: done
```

Writing inode tables: done

Creating journal (16384 blocks): done

Writing superblocks and filesystem accounting information: done

- **EXPLANATION:** Initializes block groups, reserves inode tables, writes the journal space, and configures the primary superblock.
- **ALTERNATIVE:** `mkfs.xfs /dev/sdb1` (for enterprise XFS deployments).
- **WHEN TO USE:** Immediately after partitioning new disks to prepare them for mounting.
- **SAFETY:** [!] **WARNING:** This command irrevocably formats the target partition. Any preexisting data blocks on the partition will be lost.

COMMAND: `mount`

- **PURPOSE:** Attaches an initialized storage filesystem to a specified directory location (mount point) in the FHS tree.
- **SYNTAX:** `mount [options] <device> <mount_point>`
- **EXAMPLE:**
Bash
`sudo mkdir -p /mnt/newdisk`
`sudo mount /dev/sdb1 /mnt/newdisk`
- **EXPECTED OUTPUT:** Returns cleanly to the command prompt without stdout upon successful mounting.
- **ALTERNATIVE:** Add entry to `/etc/fstab` for automated persistent mounting across system reboots.
- **WHEN TO USE:** Integrating storage partitions, network shares, or ISO images into the visible directory layout.

COMMAND: `df`

- **PURPOSE:** Reports operational disk space usage across all mounted filesystems.
- **SYNTAX:** `df [options] [path]`
- **BREAKDOWN:**
—

- - df → Disk Free utility.
- - h → Formats storage metrics using human-readable units (K, M, G, T).
- - T → Prints the underlying filesystem type column.
- - EXAMPLE:**
Bash
df -h
- - EXPECTED OUTPUT:**
Filesystem Size Used Avail Use% Mounted on
devtmpfs 700M 0 700M 0% /dev
tmpfs 730M 0 730M 0% /dev/shm
/dev/mapper/cl-root 19G 4.4G 15G 23% /
/dev/sda1 1014M 242M 773M 24% /boot
/dev/sdb1 3.9G 16M 3.7G 1% /mnt/newdisk
- - EXPLANATION:** Shows partition storage capacities, allocated blocks, available space, utilization percentages, and mount paths.

COMMAND: du

- - PURPOSE:** Estimates and summarizes directory space allocation on storage media.
- - SYNTAX:** du [options] [path]
- - BREAKDOWN:**
 - - du → Disk Usage analyzer.
 - - s → Summarizes total directory consumption without listing every sub-item.
 - - c → Produces a grand total summary line at the end.
 - - h → Formats capacity numbers in human-readable notation (e.g., MB, GB).
- - EXAMPLE:**
Bash
du -sch /var/log
- - EXPECTED OUTPUT:**
142M /var/log

- 142M total

- EXPLANATION:** Traverses the target hierarchy, aggregates used blocks, and prints the total storage space consumed on disk.

PART 5 — FILE & DIRECTORY MANIPULATION

5.1 The "Everything is a File" Paradigm

Under Linux, the operating system treats almost all system entities—text documents, directories, physical hard drives, process memory maps, network sockets, and terminal consoles—as streams of bytes exposed through standard file operations.

```
+-----+-----+-----+
| Linux Entity | Underlying Abstraction | Filesystem Path Representation |
+-----+-----+-----+
| Directory | File listing names/IDs | /home/karuna, /etc |
| Hard Disk / LUN | Block Device File | /dev/sda, /dev/sdb1 |
| Pseudo Terminals | Character Device File | /dev/pts/0, /dev/tty1 |
| Kernel Metrics | Dynamic Virtual File | /proc/cpuinfo, /proc/meminfo |
| Hardware Monitor | Device Interface File | /dev/null, /dev/zero, /dev/urandom |
+-----+-----+-----+
```

5.2 Core Directory & File Management Commands

COMMAND: pwd

- PURPOSE:** Outputs the full absolute path of the present working directory.

- SYNTAX:** pwd [options]

- EXAMPLE:**

```
Bash
pwd
```

- EXPECTED OUTPUT:**

```
/home/karuna/SamplePrograms
```

- EXPLANATION:** Evaluates the active process's current working directory context and prints the path from the root directory.

COMMAND: cd

-

- **PURPOSE:** Changes the shell's active working directory context.

- **SYNTAX:** `cd [target_path]`

- **SPECIAL DIRECTORY SHORTCUTS:**

- `.` → Current directory.

- `..` → Immediate parent directory.

- `~` → Active user's home directory.

- `-` → Previous working directory.

- **EXAMPLES:**

Bash

`cd /var/log` # Absolute Path Navigation (Begins at root /)

`cd ../etc` # Relative Path Navigation (Relative to current position)

`cd ~` # Return directly to user's home directory

`cd -` # Switch back to the previous directory

- **COMMON MISTAKE:** Typing `cd..` without a space between the command and argument, which results in a `bash: cd..: command not found` error.

COMMAND: `ls`

- **PURPOSE:** Lists directory contents and displays detailed file metadata.

- **SYNTAX:** `ls [options] [target_path]`

- **OPTIONS:**

- `-l` → Long format display (permissions, links, owner, group, size, timestamps).

- `-a` → Includes hidden files (entries starting with `.`).

- `-t` → Sorts entries chronologically by modification time.

- `-h` → Formats file sizes with human-readable units (used with `-l`).

- `-d` → Lists the directory entry itself rather than its contents.

EXAMPLE:

```
Bash
ls -la
```

COMMAND: mkdir

- **PURPOSE:** Creates one or more new directories at the specified path.
- **SYNTAX:** mkdir [options] <directory_name>
- **OPTIONS:**
 - **-p** → Creates parent directories as needed without throwing errors if they already exist.
- **EXAMPLE:**

```
Bash
mkdir -p /home/karuna/projects/src/module1
```

COMMAND: touch

- **PURPOSE:** Updates file access and modification timestamps; creates an empty file if the target does not exist.
- **SYNTAX:** touch <filename>
- **EXAMPLE:**

```
Bash
touch sample_script.sh
```

COMMAND: cp

- **PURPOSE:** Copies files and directory trees from source to destination paths.
- **SYNTAX:** cp [options] <source> <destination>
- **OPTIONS:**
 - **-r, -R** → Recursively copies directories and their contents.
 - **-p** → Preserves original file attributes, timestamps, and ownership modes.
 - **-i** → Prompts for confirmation before overwriting existing files.

- **EXAMPLE:**
Bash
`cp -r /home/karuna/SamplePrograms /home/karuna/BackupPrograms`

COMMAND: mv

- **PURPOSE:** Moves or renames files and directories within the filesystem.
- **SYNTAX:** `mv [options] <source> <destination>`
- **EXAMPLE:**
Bash
`mv old_benchmark.py new_benchmark.py # Renames file`
`mv new_benchmark.py ./production/ # Relocates file into directory`

COMMAND: rm

- **PURPOSE:** Unlinks and deletes specified files or directory hierarchies.
- **SYNTAX:** `rm [options] <target>`
- **OPTIONS:**
 - `-r` → Recursively removes directories and their contents.
 - `-f` → Force deletion without prompting or reporting non-existent targets.
 - `-i` → Interactively prompts before every file deletion.
- **SAFETY: [!] WARNING:** Linux CLI lacks a trash bin or default undo mechanism. Running destructive commands such as `rm -rf /` or `rm -rf *` can permanently erase critical system and user data. Always verify the current path with `pwd` before using recursive force options.

COMMAND: file

- **PURPOSE:** Examines file headers to determine file type, independent of file extensions.
- **SYNTAX:** `file <filename>`
- **EXAMPLE:**
Bash

file sample_data

EXPECTED OUTPUT:

sample_data: ASCII text executable

EXPLANATION: Inspects magic numbers in file headers rather than relying on file extensions to identify file contents.

5.3 Shell Wildcard Pattern Matching

The shell parses wildcard patterns to match multiple filenames in bulk operations:

Pattern	Match Rule Definition	Example Usage
*	Matches zero or more arbitrary characters	ls *.log (lists all .log files)
?	Matches exactly one single character	cat run_?0.sh (matches run_10.sh)
[chars]	Matches any single character in brackets	ls data_[abc].csv
[a-z]	Matches any character within the range	rm temp_[0-9].txt
[!chars]	Matches any character not enclosed in brackets	ls data_[!0-9].csv

PART 6 — TEXT DISPLAY, INSPECTION, AND PATTERN SEARCHING

6.1 Text Display Commands

TEXT DISPLAY SELECTION DECISION MATRIX			
Requirement	Recommended Command	Memory Footprint	Interactive Scroll
Small Files	cat	Loads full file	No (Prints to STDOUT)
Reverse File	tac	Loads full file	No (Reversed lines)
Large Files	less	On-demand pages	Yes (Full movement)
Screen-page	more	Forward paging	Forward only
File Header	head -n <lines>	Minimal / Line-cap	No
File Footer	tail -n <lines>	Minimal / Line-cap	No
Live Logs	tail -f	Continuous polling	Real-time monitoring

COMMAND: cat

PURPOSE: Concatenates and prints file contents directly to standard output.

- **SYNTAX:** `cat [options] [files]`
- **OPTIONS:**
 - `-n` → Numbers all output lines.
 - `-b` → Numbers non-empty output lines.
- **EXAMPLE:**
Bash
`cat /etc/resolv.conf`

COMMAND: tac

- **PURPOSE:** Concatenates and prints files in reverse order (bottom line first).
- **SYNTAX:** `tac <filename>`
- **EXAMPLE:**
Bash
`tac execution.log`

COMMAND: less

- **PURPOSE:** Terminal pager utility for viewing files interactively with bidirectional scrolling.
- **SYNTAX:** `less <filename>`
- **INTERACTIVE NAVIGATION COMMANDS:**
 - `Spacebar / f` → Advance forward one full screen page.
 - `b` → Move backward one full screen page.
 - `g` → Jump directly to the first line of the file.
 - `G` → Jump directly to the last line of the file.
 - `/pattern` → Search forward for a specific string pattern.
 - `n` → Repeat search forward for next occurrence.
 -

- **q** → Exit pager and return to shell prompt.

- **WHY USE OVER CAT:** cat dumps the entire file into the terminal scrollbar buffer at once, which can freeze sessions on multi-gigabyte datasets. less loads only the visible screen buffer, keeping memory usage minimal regardless of file size.

COMMAND: head

- **PURPOSE:** Prints the first lines of a specified file or input stream.

- **SYNTAX:** head -n <number_of_lines> <filename>

- **EXAMPLE:**

Bash

head -n 5 /etc/passwd

COMMAND: tail

- **PURPOSE:** Prints the ending lines of a specified file or stream.

- **SYNTAX:** tail [options] <filename>

- **OPTIONS:**

- **-n <num>** → Outputs the last specified number of lines.

- **-f** → Keeps the file open and follows changes, printing new lines in real time as they are appended.

- **EXAMPLE:**

Bash

tail -f /var/log/messages

6.2 Advanced Pattern Searching: grep

1. Definition

grep (Global Regular Expression Print) searches files or standard input for lines matching a specified pattern and prints the matching text to standard output.

2. Syntax

Bash

```
grep [options] "search_pattern" <file_or_directory_targets>
```

3. Options

- `-i` → Performs case-insensitive matching.
- `-r, -R` → Recursively searches through all subdirectories under target directories.
- `-I` → Automatically ignores and skips binary files to prevent console corruption.
- `-v` → Inverts matching, printing only lines that do not match the pattern.
- `-n` → Prints the matching line numbers alongside output text.
- `-c` → Suppresses normal line output, printing only the count of matched lines.
- `-E` → Treats the search string as an Extended Regular Expression (ERE).

4. Examples

Bash

```
# Search for case-insensitive occurrences of "error" across text logs
grep -i -r --exclude-dir=proc "error" /var/log/*.log
```

```
# Search using pipe inputs
ps -aef | grep mysql
```

6.3 Complex Filesystem Searching: find

1. Definition

The find utility traverses directory trees in real time, searching for files based on attributes such as filename patterns, types, sizes, modification times, and ownership.

2. Syntax

Bash

```
find <starting_path> [criteria_options] [actions]
```

3. Core Search Criteria Flags

- `-name <pattern>` → Matches file base name using shell wildcards.

- `-iname <pattern>` → Case-insensitive version of `-name`.
- `-type <t>` → Filters by object type: `f` (regular file), `d` (directory), `l` (symlink), `c` (character device), `b` (block device).
- `-mtime <n>` → Filters by modification time in days (`-7` = within 7 days, `+30` = older than 30 days).
- `-size <n>` → Matches file size (`+100M` = larger than 100MB, `-1G` = smaller than 1GB).
- `-exec <cmd> {} \;` → Executes the specified command on each matched file, replacing `{}` with the file path.

4. Examples

Bash

Locate all test files in current directory tree

```
find . -type f -name "*test*"
```

Locate all system files modified within the last 24 hours and format list

```
find /home/karuna -type f -mtime -1 -exec ls -l {} \;
```

PART 7 — INPUT/OUTPUT STREAMS, REDIRECTION, & COMMAND PIPELINES

7.1 Linux Standard Streams Architecture

In Linux, standard I/O streams are handled via three default file descriptors opened for every active process:

```
+-----+
| PROCESS STREAM COMMUNICATIONS |
| |
| +-----+ |
| Standard Input (stdin) | | ---> Standard Output (stdout) |
| [File Descriptor: 0] ---> | PROCESS | [File Descriptor: 1] |
| (Keyboard / Input Stream) | | ---> Standard Error (stderr) |
| +-----+ [File Descriptor: 2] |
| |
+-----+
+-----+-----+-----+-----+
| Stream | File Descriptor | Default Source | Common Redirection Targets |
```

```
+-----+-----+-----+-----+
| stdin | 0 | Keyboard | Files, input heredocs, pipes |
| stdout | 1 | Terminal Screen | Log files, null sink, pipes |
| stderr | 2 | Terminal Screen | Error logs, audit files |
+-----+-----+-----+-----+
```

7.2 Redirection Operators

- `>`: Redirects standard output (file descriptor 1) to a file, creating it or overwriting existing contents.
- `>>`: Appends standard output to a target file without overwriting existing data.
- `<`: Redirects standard input (file descriptor 0) from an existing file instead of the terminal keyboard.
- `2>`: Redirects standard error output (file descriptor 2) to a target file.
- `2>&1`: Merges file descriptor 2 (stderr) into file descriptor 1 (stdout), combining both streams into a single destination.
- `&>`: Bash shortcut that redirects both stdout and stderr into the same target file.

Bash

```
# Split stdout and stderr into separate log files
calculate_mpi_job > output.log 2> error.log
```

```
# Merge stdout and stderr into a single log file
compile_kernel > build.log 2>&1
```

```
# Suppress stderr messages completely
find / -name "hpc_job.sh" 2> /dev/null
```

7.3 Pipelines (|)

The pipe operator (`|`) connects the standard output (stdout) of an upstream command directly to the standard input (stdin) of a downstream command without writing intermediate data to disk.

```
+-----+ stdout +-----+
| command1 | =====> | command2 |
| (e.g., ps -aef) | (The Pipe |) | (e.g., grep root) |
+-----+ +-----+
```

Bash

```
# Filter active processes to find the MariaDB daemon
ps -aef | grep mysql
```

```
# Count unique logged-in users
who | awk '{print $1}' | sort | uniq | wc -l
```

PART 8 — MULTI-USER SECURITY, PERMISSIONS, & ACCESS CONTROL

8.1 Discretionary Access Control (DAC) Model

Linux controls file access using a multi-user model where permissions are evaluated across three user classes:

```

rwxr-xr--
+----+ +----+ +----+
| User | |Group| |Other|
+----+ +----+ +----+
Owner Class Group Class World Class
+-----+-----+-----+-----+
| Class | Identity Match | Security Scope & Application |
+-----+-----+-----+-----+
| User | Owner ('u') | Applies to the specific user account that owns |
| | | the file. |
| Group | Group Owner ('g') | Applies to accounts belonging to the file's |
| | | assigned group. |
| Other | World / Rest ('o') | Applies to all other accounts on the system. |
+-----+-----+-----+-----+

```

8.2 Permission Bit Semantics

Permissions have distinct meanings depending on whether they are applied to files or directories:

```

+-----+-----+-----+-----+-----+
| Permission | Letter | Octal | On Regular Files | On Directories |
+-----+-----+-----+-----+-----+
| Read | r | 4 | View file data | List directory entries ('ls') |
| Write | w | 2 | Modify file data | Create, delete, rename files |
| Execute | x | 1 | Run file as program | Traverse directory ('cd') |
+-----+-----+-----+-----+-----+

```

8.3 Long Listing Metadata Anatomy (ls -l)

```

-rwxr-xr-- 1 shum staff 4096 Jan 16 14:07 benchmark.sh
| \_/_/ \_/_/ \_/_/ | | | | | |

```

```

| | | | | | | | +-> File Name
| | | | | | | | +-----> Last Modified
| | | | | | | | +-----> Size in Bytes
| | | | | | | | +-----> Group Owner
| | | | | | | | +-----> User Owner
| | | | | | | | +-----> Hard Link Count
| | | | | | | | +-----> SELinux Dot
| | | | | | | | +-----> Other Perms
| | | | | | | | +-----> Group Perms
| | | | | | | | +-----> Owner Perms
+-----> File Type
(-: regular,
d: directory,
l: symlink)

```

8.4 Permission Modification Commands

COMMAND: chmod

- PURPOSE:** Modifies the discretionary access permissions of files or directories.
- SYNTAX:**
 - Symbolic Mode: `chmod [who][operation][permissions] <target>`
 - Numeric (Octal) Mode: `chmod <octal_mode> <target>`
- OCTAL CALCULATION:**

$$\text{Value} = \text{Read}(4) + \text{Write}(2) + \text{Execute}(1)$$
 - $\$7 = 4 + 2 + 1\$$ (Full Read, Write, Execute)
 - $\$6 = 4 + 2 + 0\$$ (Read and Write)
 - $\$5 = 4 + 0 + 1\$$ (Read and Execute)
 - $\$4 = 4 + 0 + 0\$$ (Read Only)
 - $\$0 = 0 + 0 + 0\$$ (No access permissions)
- EXAMPLES:**

```

Bash
# Octal Mode: Owner: rwx (7), Group: r-x (5), Other: --- (0)

```

```
chmod 750 run_simulation.sh
```

```
# Symbolic Mode: Add execute to user owner, remove write from group
```

```
chmod u+x,g-w data_process.py
```

COMMAND: chown

- **PURPOSE:** Changes the user owner and group owner of a file or directory.
- **SYNTAX:** `chown [options] <new_owner>[:<new_group>] <target>`
- **EXAMPLES:**
 Bash
 # Modify user owner only
`sudo chown karuna dataset.csv`

 # Modify both user owner and group owner simultaneously
`sudo chown karuna:hpcusers dataset.csv`

 # Recursively change ownership for an entire directory tree
`sudo chown -R karuna:hpcusers /opt/benchmarks`

8.5 The umask Architecture

The umask (user mask) controls the default permissions assigned to newly created files and directories. Default base permissions are subtracted using the umask filter:

- System Base Permissions:
 - Directories: 777 (rwxrwxrwx)
 - Regular Files: 666 (rw-rw-rw-)
- Calculating permissions:

$$\text{Final Default Permission} = \text{Base Permission} - \text{umask}$$

Directories: 777 (rwxrwxrwx) Regular Files: 666 (rw-rw-rw-)

umask 022: - 022 (---w--w-) umask 022: - 022 (---w--w-)

Result: 755 (rwxr-xr-x) Result: 644 (rw-r--r--)

8.6 Special Permissions: SUID, SGID, and Sticky Bit

Permission	Octal	Flag Bit	Applied to Files	Applied to Directories
SUID	4000	s in user	Executes with privileges	No operational effect
(SetUID)		position	of the file owner	
		(e.g., /usr/bin/passwd)		
SGID	2000	s in group	Executes with group	New files inherit
(SetGID)		position	privileges of file; group ownership	
			processes run as group	the parent directory
Sticky Bit	1000	t in other	No operational effect	Only file owner or root can delete files
		position	on modern systems	
				in the directory

- **SUID (Set Owner User ID):** When an executable with SUID runs, the process runs with the privileges of the file's owner rather than the invoking user.

Bash

Example: Setting SUID bit on an executable
 sudo chmod u+s /usr/local/bin/custom_tool

In ls -l appears as: -rwsr-xr-x

- **SGID (Set Group ID):** When set on a shared directory, all newly created files and subdirectories inherit the group ownership of that parent directory instead of the creator's primary group.

Bash

Example: Shared collaborative directory
 sudo chmod g+s /shared/hpc_project

In ls -l appears as: drwxr-sr-x

- **Sticky Bit:** Applied to shared writeable directories (such as /tmp). While all users can write files, users can only delete or rename files they personally own.

Bash

Example: Protecting a public directory
 sudo chmod o+t /tmp/scratch_space

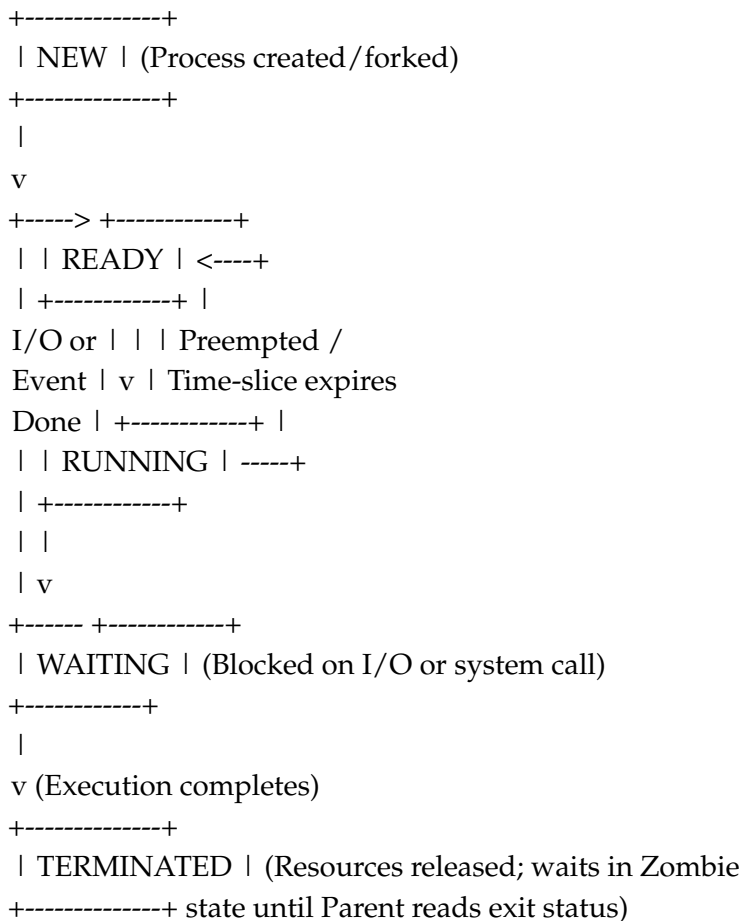
In ls -l appears as: drwxrwxrwt

PART 9 — PROCESS MANAGEMENT, CPU SCHEDULING, & SYNCHRONIZATION

9.1 Process Concept and Lifecycle States

1. Definition

A **process** is an active instance of a program in execution, encompassing program code, allocated system resources, memory maps, and an assigned Process ID (PID) managed by the operating system kernel.



2. Process Control Block (PCB)

The **Process Control Block (PCB)** is the kernel data structure used to track, schedule, and isolate a process:

- **Process Identification (PID):** Unique numeric identifier for the process.
- **Process State:** Current execution state (Ready, Running, Waiting, Suspended, Zombie).
- **Program Counter (PC):** Stores the address of the next instruction to execute.
- **CPU Registers:** Saves register state (accumulators, stack pointer, index registers) during context switches.
-

Memory Management Information: Base/limit registers, page tables, and virtual memory maps.

- **Accounting Information:** CPU execution time, time limits, and parent process ID (PPID).
- **I/O Status Information:** List of open file descriptors and assigned I/O devices.

3. Context Switching

A **Context Switch** occurs when the CPU scheduler stops a running process, stores its execution state into its PCB, and restores another process's state from its respective PCB.

- *Overhead:* During a context switch, the CPU performs administrative state saves and restorations rather than executing application code. Excessive context switches can degrade overall system throughput.

9.2 CPU Scheduling Algorithms

CPU scheduling algorithms determine how the operating system assigns CPU execution time to processes waiting in the ready queue.

Algorithm	Category	Scheduling Metric / Selection Strategy
FCFS	Non-preemptive	First Come, First Served. Simple FIFO queue (First Come) order; prone to convoy effect delays.
SJF	Non-preemptive	Shortest Job First. Selects smallest burst (Shortest Job) time; optimal average wait, risks starvation
SRTF	Preemptive	Shortest Remaining Time First. Preempts if (Shortest Remain) a new process has a shorter remaining burst
Round Robin (RR)	Preemptive	Allocates fixed CPU time quantum; cycles through processes evenly; fair and stable
Priority Scheduling	Preemptive or Non-preemptive	Highest priority process runs first; lower priority jobs may suffer from starvation
HRRN	Non-preemptive	Highest Response Ratio Next. Balances burst time and wait time to prevent starvation

$$\text{Response Ratio} = \frac{\text{Waiting Time} + \text{Burst Time}}{\text{Burst Time}}$$

9.3 Process Synchronization

1. Theory

When multiple processes or threads concurrently read and write shared memory or files, operations can interleave unpredictably, resulting in a **race condition**. Operating systems use synchronization primitives to coordinate access to this shared **critical section**:

- **Mutex (Mutual Exclusion):** A locking mechanism ensuring that only one thread or process can enter a critical section at a time.
- **Semaphores:** Integer variables accessed via atomic operations:
 - **wait() (or P):** Decrements the semaphore counter; blocks if the value is ≤ 0 .
 - **signal() (or V):** Increments the semaphore counter, waking waiting tasks.
 - **Binary Semaphore:** Values are limited to 0 and 1, functioning similarly to a mutex lock.
 - **Counting Semaphore:** Tracks access across a pool of identical available resources.
- **Deadlock Conditions:** A state where processes are permanently blocked because each holds a resource while waiting for another resource held by another process.

9.4 Process Inspection & Management Commands

COMMAND: ps

- **PURPOSE:** Reports snapshots of active system processes.
- **SYNTAX:** ps [options]
- **COMMON VARIANTS:**
 - ps aux → BSD syntax showing all running processes across all users with resource utilization.
 - ps -ef, ps -aef → Standard Unix syntax listing full process trees with parent IDs (PPID).
- **EXAMPLE:**
Bash
ps aux | grep python

COMMAND: top

- **PURPOSE:** Provides a real-time, dynamic view of running system processes, CPU utilization, memory allocations, and system load.
- **SYNTAX:** top
- **INTERACTIVE HOTKEYS INSIDE top:**
 - u → Filters visible processes by a specific username.
 - k → Prompts for a PID to terminate (sends a kill signal).
 - h → Opens the interactive help screen.
 - f → Field management (customizes visible metrics columns).
 - x → Toggles column highlighting on the active sort column.
 - < and > → Moves the active sort column left or right.
 - R → Reverses the current sort order (ascending/descending).
 - q → Quits the utility.

COMMAND: pstree

- **PURPOSE:** Displays currently running processes as an ASCII tree structure, showing parent-child process relationships.
- **SYNTAX:** pstree [options] [PID/user]
- **EXAMPLE:**
Bash
pstree -p

COMMAND: kill

- **PURPOSE:** Transmits termination or operational control signals to processes by PID.
- **SYNTAX:** kill [signal_option] <PID>
- **CORE KERNEL SIGNALS:**

- | -1 (SIGHUP) → Hangup; reloads service configuration files without full restarts.
- | -2 (SIGINT) → Interrupt signal; equivalent to pressing Ctrl-C.
- | -9 (SIGKILL) → Force kill; immediately terminates the process at the kernel level. Cannot be caught or ignored.
- | -15 (SIGTERM) → Standard graceful termination request (default signal).

9.5 Job Control: Foreground and Background Execution

- | **Run in Background (&):** Appending & to a command runs it in the background, freeing the terminal prompt for immediate use.
Bash
python3 heavy_simulation.py > sim.log 2>&1 &
- | **Suspend Current Task (Ctrl-Z):** Halts execution of the current foreground process and moves it into a paused background state.
- | **Resume in Background (bg):** Resumes a suspended job in the background.
Bash
bg %1
- | **Bring to Foreground (fg):** Re-attaches a background job to the active terminal session.
Bash
fg %1
- | **List Active Shell Jobs (jobs):** Lists background jobs running under the current shell session, displaying their job IDs.

PART 10 — OPERATING SYSTEM MEMORY MANAGEMENT

10.1 The Memory Hierarchy

Operating systems balance access latency and storage costs using a tiered memory hierarchy:

```

/\
/\
/ L1 \ Registers & L1 Cache (~0.5 - 1 ns)
/-----\
/ L2 \ L2 Cache (~3 - 7 ns)

```

```

/-----\
/ L3 \ L3 Cache Shared On-Die (~10 - 20 ns)
/-----\
/ Main Memory \ DRAM Physical RAM (~50 - 100 ns)
/-----\
/ Solid State \ NVMe / Flash Storage (~10 - 50 us)
/-----\
/ Mechanical Disk \ Mechanical Hard Drives (~5 - 10 ms)
+-----+

```

10.2 Logical vs. Physical Address Spaces & The MMU

- **Logical (Virtual) Address Space:** An abstracted memory address range generated by the CPU for an executing process. Virtual memory isolates processes into uniform, contiguous address spaces.
- **Physical Address Space:** The actual physical memory locations in RAM modules.
- **Memory Management Unit (MMU):** A hardware component on the CPU that translates virtual addresses into physical addresses on the fly using page tables.

```

+-----+
| CPU Core Running |
| Program Code |
+-----+
|
| (Generates Virtual Logical Address)
v
+-----+
| Memory Management | ===== Translates via =====> [ System Page Table ]
| Unit (MMU) | [ (Virtual -> Physical) ]
+-----+
|
| (Emits Translated Physical Address)
v
+-----+
| Physical Main Memory (RAM) |
+-----+

```

10.3 Paging and Virtual Memory

1. Theory: Paging Architecture

-

Pages and Frames: Virtual memory divides a process's address space into fixed-size blocks called **pages** (typically 4 KB in size). Physical RAM is similarly divided into matching fixed-size blocks called **frames**.

- **Page Tables:** Maintained by the kernel for every running process, mapping virtual page numbers to corresponding physical frames in RAM.

- **Page Fault Handling:** If a process requests a memory page that is not currently loaded in physical RAM, the MMU triggers a **page fault** interrupt. The kernel suspends the process, loads the missing page from disk swap space into an available physical RAM frame, updates the page table flags, and resumes process execution.

2. Segmentation vs. Paging

+-----+-----+-----+		
Feature	Paging Architecture	Segmentation Architecture
+-----+-----+-----+		
Block Dimensions	Fixed size (e.g., 4KB, 2MB, 1GB)	Variable size based on program
	determined by hardware	logical components
Programmer View	Completely invisible; managed	Aligns with code structure
	by MMU hardware and kernel	functions, arrays, stack
Fragmentation	Zero external fragmentation;	Suffers from external
	small internal fragmentation	fragmentation over time
Translation Table	Page Table (Page # -> Frame #)	Segment Table (Base + Limit)
+-----+-----+-----+		

3. Swapping

When physical RAM is exhausted under heavy multitasking, the operating system kernel swaps idle memory pages to a designated disk partition or swap file. This frees physical RAM frames for active tasks, swapping data back when those processes are rescheduled.

PART 11 — LINUX PACKAGE MANAGEMENT SYSTEMS

11.1 Package Management Architecture

+-----+	
Remote Software Repos	
(HTTP/S mirrors of .deb/	
.rpm software packages)	
+-----+	
(Sync via apt update /	

```

| yum/dnf makecache)
v
+-----+ +-----+
| User CLI Requests | ==> | Local Package Manager |
| (apt install / dnf in) | | (Resolves dependencies, |
+-----+ | downloads required libs) |
+-----+
|
| (Calls low-level installer)
v
+-----+
| Low-Level Package Engine |
| (dpkg or rpm) |
+-----+
|
v
+-----+
| Host System Storage |
| (/bin, /lib64, /etc) |
+-----+

```

A **Package Manager** automates installing, configuring, upgrading, querying, and removing software packages on Linux systems.

- **Package Contents:** An archive holding executable binaries, default configurations, dependency manifests, and installation scripts.
- **Repositories:** Centralized remote network servers hosting signed software packages and metadata catalogs.
- **Dependency Resolution:** Identifies prerequisites and installs required libraries automatically to prevent installation failures.

11.2 Package Architecture Comparison

```

+-----+ +-----+ +-----+ +-----+
| Feature Scope | Debian Family | Red Hat Enterprise | Red Hat Modern |
| | (Debian, Ubuntu, Mint) | (RHEL 7, CentOS 7) | (RHEL 8/9, Fedora) |
+-----+ +-----+ +-----+ +-----+
| High-Level Tool | APT ('apt-get', 'apt') | YUM ('yum') | DNF ('dnf') |
| Low-Level Tool | 'dpkg' | 'rpm' | 'rpm' |
| Package Extension | '.deb' | '.rpm' | '.rpm' |
| Dependency Engine | Fully Automatic | Fully Automatic | Automatic (Hawkey) |
| Relative Speed | Fast | Moderate | Fast (C-based solver) |

```

+-----+-----+-----+-----+

11.3 Package Management Commands

1. Debian / Ubuntu Systems (APT & dpkg)

```
Bash
# Update local repository metadata cache
sudo apt-get update

# Install software package and resolve dependencies
sudo apt-get install nginx

# Upgrade all installed packages to latest compatible versions
sudo apt-get upgrade

# Remove package while preserving user configuration files
sudo apt-get remove nginx

# Purge package along with all configuration files
sudo apt-get purge nginx

# Query local database for packages matching keyword
apt-cache search openmpi

# Low-Level dpkg: Install standalone local .deb package
sudo dpkg -i package_name.deb
```

2. Enterprise Red Hat / CentOS / Fedora Systems (RPM, YUM, DNF)

```
Bash
# Update local repository metadata cache
sudo dnf makecache

# Install software package from active repositories
sudo dnf install httpd

# Upgrade all system packages
sudo dnf upgrade

# Remove software package and orphan dependencies
sudo dnf remove httpd

# Low-Level RPM Operations:
```

```
rpm -i package.rpm # Install individual package directly
rpm -U package.rpm # Upgrade existing package directly
rpm -q httpd # Query if package is installed
rpm -V httpd # Verify package integrity against checksums
rpm -e httpd # Erase / uninstall package
```

PART 12 — THE VIM TEXT EDITOR

12.1 The Modal Editing Philosophy

Vim (*Vi IMproved*) operates using distinct operational modes, separating text entry from cursor navigation and text manipulation.

```
Press i, a, or o
+-----> +-----+
| | INSERT MODE |
| <Esc> (Return to default state) | (Type text data) |
| <-----+-----+
|
+-----+
| NORMAL MODE |
| (Navigation & ops) |
+-----+
|
| Press : (Colon prompt) +-----+
+-----> | COMMAND MODE |
| | (Save, quit, regex) |
| <Esc> or <Enter> +-----+
+ <-----+
```

- **Normal Mode:** Default state on startup. Keystrokes are interpreted as navigation shortcuts or manipulation commands rather than literal text.
- **Insert Mode:** Active editing state for typing text into the document buffer. Entered using i, a, or o. Return to Normal Mode by pressing Esc.
- **Command-Line (Ex) Mode:** Initiated by typing : from Normal Mode. Used for saving files, exiting, configuring editor settings, and running global search-and-replace patterns.

12.2 Vim Command Reference

```

+-----+
| VIM KEYBOARD SHORTCUT COMPENDIUM |
+-----+
| Context | Keystroke & Operational Effect |
+-----+
| Switch | i -> Enter Insert Mode at cursor position |
| Modes | a -> Enter Insert Mode after cursor position |
| | o -> Open new line below cursor and enter Insert Mode |
| | Esc -> Exit back to Normal Mode |
| | : -> Enter Command-Line Mode |
+-----+
| Normal | h, j, k, l -> Move cursor left, down, up, right |
| Motion | w -> Jump forward to start of next word |
| | b -> Jump backward to previous word |
| | $ -> Move to end of current line |
| | 0 -> Move to beginning of current line |
| | gg -> Jump to first line of file |
| | G -> Jump to last line of file |
| | % -> Jump to matching parenthesis, brace, or bracket |
+-----+
| Buffer | u -> Undo last change |
| Editing | Ctrl-r -> Redo last undone change |
| | . -> Repeat the last edit command |
| | x -> Delete (cut) character under cursor |
| | dd -> Delete (cut) current line |
| | yy (or Y) -> Yank (copy) current line |
| | p -> Paste copied/deleted buffer after cursor |
+-----+
| Search & | /pattern -> Search forward for regular expression |
| Replace | ?pattern -> Search backward for regular expression |
| | n -> Repeat last search in same direction |
| | N -> Repeat last search in opposite direction |
| | :%s/old/new/g -> Replace all occurrences in entire file |
| | :%s/old/new/gc -> Replace with confirmation prompts |
+-----+
| Save & | :w -> Save buffer to disk |
| Quit | :w name -> Save buffer as a new file name |
| | :wq / :x / ZZ -> Save changes and exit Vim |
| | :q! -> Discard changes and force exit |
+-----+

```

13.1 Shell Script Fundamentals & The Shebang

A **Shell Script** is an executable text file containing a sequence of commands executed step by step by a designated shell interpreter, enabling task automation and system management.

1. Anatomy of a Shell Script

```
Bash
#!/bin/bash
# The line above is the Shebang (# and !), defining the execution interpreter.
# Comments begin with a hash character (#) and are skipped by the parser.

echo "Beginning System Health Check..."
echo "Current Date and System Time: $(date)"
```

- **The Shebang (!):** Must be placed on the very first line of the script. It provides the absolute path to the binary interpreter used to run the code (e.g., `#!/bin/bash`, `#!/bin/sh`, `#!/usr/bin/python3`).
- **Script Permissions:** Scripts must have execute permissions to run directly as programs:

```
Bash
chmod u+x run_check.sh
./run_check.sh
```
- **Why ./ is required:** For security reasons, Linux does not include the current working directory (.) in the default search \$PATH. Specifying ./ provides an explicit relative path to the local script.

13.2 Shell Quoting Rules

Quote Type	Notation	Expansion / Parsing Behavior
Single Quotes	'literal string'	Preserves literal value of every character; variables are NOT expanded (e.g., '\$VAR' outputs \$VAR).
Double Quotes	"expanded string"	Preserves most whitespace, but expands variables and command substitutions (e.g., "\$VAR" expands value).
Command Substitution	\$(command) or `command`	Executes command in subshell and returns its output text.

13.3 Variables and Positional Parameters

```
Bash
# Variable Assignment: No spaces are allowed around the assignment (=) operator
PROJECT="ClimateModeling"
CORES=64

# Parameter Expansion
echo "Running project $PROJECT on $CORES cores."

# Positional Parameters (Arguments passed to scripts)
# Script invocation: ./deploy.sh -f 5.0 production
echo "Invoked Script Name: $0" # Outputs: ./deploy.sh
echo "First Argument: $1" # Outputs: -f
echo "Second Argument: $2" # Outputs: 5.0
echo "Third Argument: $3" # Outputs: production
echo "Total Args Passed: $# " # Outputs: 3
echo "All Arguments Array: $@" # Outputs: -f 5.0 production
```

13.4 Conditional Logic Structures

1. The if-elif-else Construct

```
Bash
#!/bin/bash
read -p "Enter processing capacity percentage: " VALUE

if [ "$VALUE" -gt 100 ]; then
echo "Error: Exceeds maximum threshold."
elif [ "$VALUE" -ge 80 ]; then
echo "Warning: High computing load detected."
elif [ "$VALUE" -gt 0 ]; then
echo "Notice: Operational parameters normal."
else
echo "Notice: System idle or non-responsive."
fi
```

2. Logical Operators in Conditionals

- **&&** (Logical AND): Both conditional expressions must evaluate to true.
- **||** (Logical OR): Succeeds if at least one conditional expression evaluates to true.

```
Bash
if [ "$CORES" -ge 16 ] && [ "$PROJECT" == "ClimateModeling" ]; then
echo "High-Performance Cluster Profile Activated."
fi
```

3. String & Numeric Comparison Operators

```
+-----+-----+-----+
| Numeric Operators | String Operators | File Inspection Operators |
+-----+-----+-----+
| -eq (Equal) | == (String Equal) | -e <path> (Path exists) |
| -ne (Not Equal) | != (String Not Equal) | -f <file> (Regular file) |
| -lt (Less Than) | -z (String is Empty) | -d <dir> (Directory exists) |
| -le (Less or Equal) | -n (String Not Empty) | -r <file> (File readable) |
| -gt (Greater) | -w <file> (File writeable) |
| -ge (Greater or Equal) | -x <file> (File executable) |
+-----+-----+-----+
```

4. The case Pattern-Matching Statement

```
Bash
#!/bin/bash
echo "Choose Environment: (1) Production (2) Staging (3) Development"
read -p "Select choice: " CHOICE

case "$CHOICE" in
1)
echo "Active profile: PRODUCTION"
;;
2)
echo "Active profile: STAGING"
;;
3)
echo "Active profile: DEVELOPMENT"
;;
*)
echo "Invalid selection entered. Aborting."
exit 1
;;
esac
```

13.5 Loop Constructs

1. The for Loop

Iterates over an explicit sequence, list of files, or numeric range.

```
Bash
# Iterating through numeric range
for i in {1..5}; do
echo "Executing Job Iteration: $i"
done

# Iterating over files matching wildcard
for logfile in /var/log/*.log; do
echo "Inspecting: $logfile"
done
```

2. The while Loop

Runs continuously as long as the test condition evaluates to true.

```
Bash
COUNT=1
while [ "$COUNT" -le 5 ]; do
echo "Polling service... Attempt: $COUNT"
((COUNT++))
sleep 1
done
```

3. The until Loop

Runs continuously as long as the test condition evaluates to false, terminating once the condition becomes true.

```
Bash
STATUS=0
until [ "$STATUS" -gt 5 ]; do
echo "Waiting for database lock to release: $STATUS"
((STATUS++))
done
```

4. Loop Control Statements

- **break:** Immediately exits the loop, jumping past the closing done statement.
- **continue:** Skips the remaining commands in the current iteration and jumps directly to the next loop cycle.

PART 14 — SHELL VARIABLES, ENVIRONMENT CONFIGURATION, & HPC MODULES

14.1 Shell Variables vs. Environment Variables

- **Local Shell Variables:** Exist only within the specific parent shell session where they were defined. They are not visible to child processes, commands, or subshells.
- **Environment Variables:** Exported to the shell's environment table using the export command. They are inherited by child processes and subshells launched from that parent session.

Parent Shell Environment

```
+-----+
| LOCAL_VAR="MyData" (Not exported, hidden to child) |
| export GLOBAL_VAR="VisibleData" |
+-----+
```

```
|
| Launches child process (e.g., ./script.sh)
```

v

Child Subshell Environment

```
+-----+
| GLOBAL_VAR is accessible ("VisibleData") |
| LOCAL_VAR is unset and undefined |
+-----+
```

Bash

```
# Print all currently defined environment variables
env
```

```
# Export a persistent environment variable
export DATA_DIR="/mnt/shared/scratch"
```

14.2 The \$PATH Environment Variable

The \$PATH variable contains a colon-separated list of filesystem directories that the shell searches to locate executable commands entered in the terminal.

```
/usr/local/bin : /usr/bin : /usr/local/sbin : /usr/sbin : /home/karuna/bin
\_____/ \_____/ \_____/ \_____/ \_____/
Path 1 Path 2 Path 3 Path 4 Path 5
```

When a user runs a command like `ls`, the shell searches these directories in order from left to right, executing the first match it finds.

Bash

```
# Inspect active execution PATH
echo $PATH
```

```
# Prepend a custom binary directory to the search PATH
```

```
export PATH="/home/karuna/bin:$PATH"
```

14.3 High-Performance Computing (HPC) Environment Modules

1. Theory: The Multi-User Software Collision Problem

On shared supercomputers and HPC clusters, hundreds of researchers require different, often conflicting versions of software, compilers, and libraries (e.g., GCC 4.8 vs GCC 11, OpenMPI 1.10 vs OpenMPI 4.1). Installing all packages into default system paths like /usr/bin would break dependencies and cause collisions.

Environment Modules solve this problem dynamically. When a module is loaded, it modifies the user's shell environment by prepending software installation paths to \$PATH, \$LD_LIBRARY_PATH, and \$MANPATH on demand.

```
[ BASE USER SHELL ]
PATH = /usr/bin:/bin (GCC 4.8)
|
| module load openmpi/gcc/64/1.10.3
v
[ DYNAMICALLY MODIFIED SHELL ]
PATH = /opt/openmpi/1.10.3/bin : /usr/bin : /bin
LD_LIBRARY_PATH = /opt/openmpi/1.10.3/lib64 : /usr/lib64
```

2. Environment Module Commands

```
Bash
# List all software packages available on the cluster
module avail

# Load a specific software and compiler stack
module load openmpi/gcc/64/1.10.3

# Display currently loaded modules in the active session
module list

# Unload a module, resetting PATH and library variables
module unload openmpi/gcc/64/1.10.3

# Purge all loaded software modules, returning to clean base environment
module purge
```

PART 15 — SECURE SHELL (SSH) & REMOTE DATA TRANSFER

15.1 Secure Shell (SSH) Protocol Architecture

SSH (Secure Shell) is an encrypted network protocol providing secure remote command-line login, remote program execution, and network tunneling over untrusted networks. It replaces insecure legacy protocols like Telnet and rlogin by encrypting sessions using asymmetric and symmetric cryptography.

```

Local Client Workstation Remote Supercomputer / Server
+-----+ +-----+
| User enters: | Encrypted Protocol | sshd Daemon Listening |
| ssh user@remote_ip | =====> | Port: 22 (or custom e.g 4422) |
+-----+ (SSH Session) +-----+
|
|
v
+-----+
| Authenticates Credentials, |
| Allocates PTY pseudo-shell |
+-----+

```

15.2 Remote Access & Configuration Commands

1. Verifying and Managing the SSH Service

```

Bash
# Verify the active operational status of the SSH daemon
systemctl status sshd

# Start the SSH daemon if it is stopped
sudo systemctl start sshd

# Enable the SSH daemon to launch automatically on system boot
sudo systemctl enable sshd

```

2. Remote Authentication

```

Bash
# Standard SSH connection (default port 22)
ssh karuna@paramutkarsh.cdac.in

# Connect using a non-standard SSH port
ssh -p 4422 tpw45@paramutkarsh.cdac.in

# Execute a single command remotely and return results locally
ssh tpw45@paramutkarsh.cdac.in "uptime; free -m"

```

15.3 Secure Remote File Transfer: scp

The scp (Secure Copy) utility copies files securely across networks using the SSH data transfer channel, maintaining identical encryption security.

1. Syntax

```
Bash
scp [options] [[user@]source_host:]source_file [[user@]dest_host:]dest_file
```

2. Core Operational Examples

```
Bash
# Copy a local file to a remote user's home directory
scp simulation_data.h5 tpw45@paramutkarsh.cdac.in:/home/tpw45/

# Copy a file to a remote system using a custom SSH port (-P)
scp -P 4422 benchmark.py tpw45@paramutkarsh.cdac.in:/home/tpw45/benchmarks/

# Recursively copy an entire directory from a remote cluster to the local machine
scp -r tpw45@paramutkarsh.cdac.in:/home/tpw45/results/ ./local_results/
```

COMPREHENSIVE PRACTICAL LAB MANUAL

LAB 1 — Terminal Basics, Prompt Dissection, & Built-in Utilities

- **Objective:** Understand terminal operations, dissect the Bash command prompt, navigate command history, and use built-in help utilities.
- **Prerequisites:** Linux CLI terminal access.
- **Commands Used:** whoami, hostname, uname -a, date, man, whatis, clear, history.
- **Step-by-Step Procedure:**
 1. Open a terminal session. Verify your current authenticated username with whoami.
 2. Display the fully qualified hostname with hostname.
 3. Inspect operating system kernel release details using uname -a.
 - 4.

- Query manual documentation for the hostname utility: `man hostname`.
- 5. Search for single-line utility descriptions: `whatis ls`.
- 6. Execute a series of simple commands, then use the <Up-Arrow> key to recall previous commands from your history buffer.
- **Expected Result:** The terminal displays your user identity, system hostname, kernel metrics, and command manual pages.
- **Troubleshooting:** If manual pages return nothing appropriate, run `sudo mandb` to generate the system manual indexing database.
- **Challenge Task:** Display the last 10 commands executed in your session without using arrow keys (`history | tail -n 10`).

LAB 2 — Directory Navigation & Path Traversals

- **Objective:** Master navigating directories using both absolute and relative paths.
- **Prerequisites:** Completion of Lab 1.
- **Commands Used:** `pwd`, `cd`, `ls -l`, `ls -a`.
- **Step-by-Step Procedure:**
 1. Print your current starting location using `pwd`.
 2. Change to the root directory using an absolute path: `cd /`.
 3. Change into the configuration directory: `cd /etc`.
 4. Navigate up one directory level using the relative parent reference: `cd ..`
 5. Return directly to your user home directory: `cd ~`.
 6. Verify you are back in your home directory: `pwd`.
- **Expected Result:** The command prompt updates cleanly, and `pwd` accurately reflects each navigation step.

- **Troubleshooting:** Ensure spaces are included between commands and path arguments (e.g., avoid typing `cd..`).

LAB 3 — Comprehensive File Management & Maintenance

- **Objective:** Create, copy, rename, move, and delete files and directory hierarchies.
- **Prerequisites:** Basic navigation skills (Lab 2).
- **Commands Used:** `mkdir`, `touch`, `cp`, `mv`, `rm`, `file`.
- **Step-by-Step Procedure:**
 1. Create a workspace directory: `mkdir -p ~/lab3_workspace/backup`.
 2. Navigate into your new workspace: `cd ~/lab3_workspace`.
 3. Create three empty files: `touch file1.txt file2.txt data.csv`.
 4. Copy `data.csv` into the backup directory: `cp data.csv ./backup/`.
 5. Rename `file1.txt` to `report.txt`: `mv file1.txt report.txt`.
 6. Verify the file type of `report.txt`: `file report.txt`.
 7. Delete `file2.txt`: `rm file2.txt`.
- **Expected Result:** Directory trees are created, files are renamed and copied cleanly, and deletions succeed without errors.
- **Troubleshooting:** If `cp` returns omitting directory, add the recursive flag `-r`.

LAB 4 — Filesystem Searching & Filtering

- **Objective:** Locate files across the filesystem based on name patterns, file types, and modification times.
- **Prerequisites:** File management skills (Lab 3).
-

- **Commands Used:** find, which, whereis.

- **Step-by-Step Procedure:**

1. Create a search directory: `mkdir -p ~/search_lab/logs`.
2. Populate sample files: `touch ~/search_lab/app_test.log`
`~/search_lab/logs/error.log`.
3. Search for files matching the pattern `*test*`:
Bash
`find ~/search_lab -type f -name "*test*"`
4. Search for binary locations for system utilities: `which useradd` and `whereis useradd`.

- **Expected Result:** find returns exact matching file paths across all subdirectories.

- **Troubleshooting:** Always quote wildcard patterns (e.g., `"*test*"`) to prevent the shell from expanding them prematurely.

LAB 5 — Text File Inspection & Pattern Searching (grep)

- **Objective:** Inspect text files and search content using regular expressions.

- **Prerequisites:** Completion of Lab 3.

- **Commands Used:** cat, less, head, tail, grep.

- **Step-by-Step Procedure:**

1. View system users: `head -n 5 /etc/passwd`.
2. Inspect the last three entries of `/etc/passwd`: `tail -n 3 /etc/passwd`.
3. Search `/etc/passwd` for the root user:
Bash
`grep -i "^root" /etc/passwd`
4. Search recursively for the string `localhost` in `/etc`:
Bash

```
grep -r -I "localhost" /etc/
```

- **Expected Result:** Terminal displays matching lines containing the search strings.
- **Troubleshooting:** Use the -I flag to skip scanning binary files and keep output clean.

LAB 6 — Pipelines & Standard Stream Redirection

- **Objective:** Redirect standard streams and chain commands using pipelines.
- **Prerequisites:** Text utilities (Lab 5).
- **Commands Used:** >, », 2>, |, wc, sort, uniq.
- **Step-by-Step Procedure:**
 1. Redirect standard output to a file: `ls -l /etc > system_etc_inventory.txt.`
 2. Append an entry to the file: `date » system_etc_inventory.txt.`
 3. Redirect errors to a separate file:
Bash
`ls /root/forbidden 2> error.log`
 4. Pipeline file listings into word count to calculate total entries:
Bash
`ls -l /etc | wc -l`
- **Expected Result:** Output redirects cleanly to specified files without dumping to the terminal screen.
- **Troubleshooting:** Remember that > overwrites target files. Always use » to append data.

LAB 7 — Managing Users, Groups, & Access Permissions

- **Objective:** Manage Discretionary Access Control (DAC) permissions, ownership, and special permission flags.
- **Prerequisites:** Administrative (sudo) access.
-

- **Commands Used:** chmod, chown, umask, ls -l.

- **Step-by-Step Procedure:**

1. Create a test file: touch test_dac.txt.
2. View initial default permissions based on current umask: ls -l test_dac.txt.
3. Update permissions to owner read/write/execute, group read, other none (740):
Bash
chmod 740 test_dac.txt
4. Verify the updated permissions: ls -l test_dac.txt.
5. Create a directory and set the Sticky Bit:
Bash
mkdir shared_scratch
chmod o+t shared_scratch
ls -ld shared_scratch

- **Expected Result:** The permissions column reflects drwxrwxr-t on the sticky directory.

- **Troubleshooting:** Changing file owners requires superuser privileges (sudo chown ...).

LAB 8 — Process Monitoring & Job Control

- **Objective:** Monitor running processes, manage background jobs, and send process termination signals.

- **Prerequisites:** Terminal access.

- **Commands Used:** ps, top, jobs, bg, fg, kill, sleep.

- **Step-by-Step Procedure:**

1. Launch a long-running sleep command in the background: sleep 500 &.
2. List active background jobs: jobs.
3. Launch a command in the foreground: sleep 300.

4. Suspend the foreground command using Ctrl-Z.
 5. Resume execution in the background: bg.
 6. Bring the job back to the foreground: fg %1.
 7. Terminate the job using Ctrl-C.
- **Expected Result:** Process states transition smoothly between Running, Stopped, Background, and Terminated.
 - **Troubleshooting:** Use kill -9 <PID> if a process fails to respond to normal termination signals (SIGTERM).

LAB 9 — The Vim Text Editor

- **Objective:** Create and edit files using Vim's modal interface.
- **Prerequisites:** Completion of Lab 3.
- **Commands Used:** vim.
- **Step-by-Step Procedure:**
 1. Open a new file: vim demo_script.sh.
 2. Press i to enter Insert Mode.
 3. Type the following lines:
Bash
#!/bin/bash
echo "Hello World"
 4. Press Esc to return to Normal Mode.
 5. Save and exit by typing :wq and pressing Enter.
 6. Reopen the file, navigate to the second line using j, yank (copy) it with yy, and paste it below with p.
 - 7.

- 8. Run a global search-and-replace: `:%s/World/Linux/g`.

- Save and exit: `:x`.

- Expected Result:** File edits save correctly, and `cat demo_script.sh` shows the updated contents.

- Troubleshooting:** If you get lost in Vim, press Esc multiple times to ensure you are in Normal Mode.

LAB 10 — Bash Shell Scripting & Control Structures

- Objective:** Write a script that processes user input, uses conditionals, and iterates using loops.

- Prerequisites:** Vim editor skills (Lab 9).

- Commands Used:** `bash`, `chmod +x`, `read`, `if`, `for`.

- Step-by-Step Procedure:**

- 1. Create a script named `process_manager.sh` using Vim.

- 2. Add the following script content:

```
Bash
#!/bin/bash
read -p "Enter a positive number: " NUM
if [ "$NUM" -gt 0 ]; then
echo "Counting up to $NUM..."
for (( i=1; i<=NUM; i++ )); do
echo "Iteration: $i"
done
else
echo "Input was not a positive integer."
fi
```

- 3. Save and exit the editor.

- 4. Make the script executable: `chmod u+x process_manager.sh`.

- 5. Execute the script: `./process_manager.sh`.

- **Expected Result:** The script validates the numeric input and loops through the count as expected.

- **Troubleshooting:** Ensure there are spaces inside the test brackets (e.g., ["\$NUM" -gt 0]).

LAB 11 — Environment Variables & PATH Configuration

- **Objective:** Define variables, export environment variables, and update \$PATH.

- **Prerequisites:** Terminal access.

- **Commands Used:** echo, export, env.

- **Step-by-Step Procedure:**

1. Define a local shell variable: CLUSTER_NAME="Param".
2. Verify it is visible in the current shell: echo \$CLUSTER_NAME.
3. Export it to the global environment: export CLUSTER_NAME.
4. Launch a child subshell: bash.
5. Verify the variable was inherited by the child shell: echo \$CLUSTER_NAME.
6. Exit the subshell: exit.
7. Add a custom directory to your search \$PATH:
Bash
export PATH="\$HOME/custom_bin:\$PATH"
echo \$PATH

- **Expected Result:** \$PATH updates cleanly with the new directory prepended to the search order.

- **Troubleshooting:** Do not leave spaces around the assignment operator (=).

LAB 12 — Secure Shell (SSH) Remote Host Access

- **Objective:** Configure, test, and authenticate SSH connections to remote servers.

- **Prerequisites:** Network access and an active SSH server.
- **Commands Used:** ip addr, ping, systemctl status sshd, ssh.
- **Step-by-Step Procedure:**
 1. Determine your local IP address: ip addr.
 2. Test network connectivity to the target remote system: ping -c 3 <remote_ip>.
 3. Verify the local SSH daemon is running: systemctl status sshd.
 4. Connect to the remote machine over SSH:
Bash
ssh username@<remote_ip>
 5. On first connection, type yes to accept the host key fingerprint.
 6. Enter the user password when prompted.
 7. Once logged in, run hostname and whoami to verify the remote session.
 8. Disconnect and return to your local machine: exit.
- **Expected Result:** The terminal prompt switches to the remote system and returns cleanly upon typing exit.
- **Troubleshooting:** If the connection times out, verify firewall rules allow incoming traffic on port 22.

LAB 13 — Secure Remote File Transfers (scp)

- **Objective:** Transfer files and directories securely between local and remote systems.
- **Prerequisites:** Working SSH access (Lab 12).
- **Commands Used:** scp, ssh.
- **Step-by-Step Procedure:**

1. Create a test payload file on your local machine:
Bash
`echo "Cluster Benchmark Data 2026" > payload.dat`
2. Transfer the file to the remote host's home directory:
Bash
`scp payload.dat username@<remote_ip>:~/`
3. Verify the file arrived on the remote system:
Bash
`ssh username@<remote_ip> "ls -l ~/payload.dat"`
4. Copy an entire directory from the remote system back to your local machine:
Bash
`scp -r username@<remote_ip>:~/remote_results ./local_backup/`

- **Expected Result:** Files transfer securely and report 100% transfer completion in the terminal.

- **Troubleshooting:** Use `-P <port>` to specify custom SSH port numbers.

LAB 14 — HPC Environment Modules & Software Stacks

- **Objective:** Query, load, and manage dynamic software environments using Environment Modules.

- **Prerequisites:** Linux cluster environment with environment-modules installed.

- **Commands Used:** module avail, module load, module list, module unload.

- **Step-by-Step Procedure:**

1. List all software modules available on the cluster:
Bash
`module avail`
2. Check default MPI compiler paths: `which mpicc 2> /dev/null` (typically returns not found).
- 3.

- Load the OpenMPI environment module:
Bash
module load openmpi/gcc/64/1.10.3
- 4. View active loaded modules:
Bash
module list
- 5. Check which mpicc again to confirm the path points to the OpenMPI directory.
- 6. Unload the module and confirm your \$PATH returns to its clean baseline state:
Bash
module unload openmpi/gcc/64/1.10.3
- **Expected Result:** Software environment variables are dynamically updated and cleanly reset without needing to restart the shell.
- **Troubleshooting:** Use module purge to unload all modules if environment paths conflict.

LAB 15 — Integrated Linux System Administration Capstone

- **Objective:** Combine storage provisioning, user administration, permissions, and automation into a single workflow.
- **Prerequisites:** Root or sudo administrative access.
- **Commands Used:** useradd, fdisk, mkfs, mount, chmod, chown, systemctl, crontab.
- **Step-by-Step Procedure:**
 1. Create a system group and add an administrative user:
Bash
sudo groupadd hpcdev
sudo useradd -m -g hpcdev clusteradmin
 2. Partition, format with Ext4, and mount a secondary disk:
Bash
sudo mkfs.ext4 /dev/sdb1
sudo mkdir -p /data/cluster
sudo mount /dev/sdb1 /data/cluster
 - 3.

Set SGID permissions on the shared mount point:

Bash

```
sudo chown root:hpcdev /data/cluster
```

```
sudo chmod 2775 /data/cluster
```

4.

Write an automated backup script in `/data/cluster/backup.sh` that archives `/var/log` using `tar -czvf`.

5.

Make the backup script executable: `sudo chmod 750 /data/cluster/backup.sh`.

Expected Result: A fully mounted, secure collaborative directory managed by SGID group inheritance and protected by proper DAC permissions.

Troubleshooting: Verify mount points persist across reboots by adding an entry to `/etc/fstab`.

FINAL SYSTEM REFERENCE & HANDBOOK COMPENDIUM

16.1 Command Reference Cheat Sheet

Command	Core Operational Purpose	Practical Syntax Example
ls	Lists directory contents and file metadata	<code>ls -la /var/log</code>
cd	Changes the present working directory	<code>cd ../projects</code>
pwd	Displays absolute path of current working directory	<code>pwd</code>
mkdir	Creates new directory paths	<code>mkdir -p /data/archive/2026</code>
touch	Creates empty file or updates access timestamps	<code>touch runtime.lock</code>
cp	Copies files and directories recursively	<code>cp -r /source /destination</code>
mv	Moves or renames filesystem targets	<code>mv old_test.py run.py</code>
rm	Deletes files and directory hierarchies	<code>rm -rf /tmp/scratch_dir</code>
cat	Concatenates and prints file streams to terminal	<code>cat output.log</code>
less	Interactive terminal pager with bidirectional scrolling	<code>less /var/log/messages</code>
head / tail	Displays starting or ending lines of files	<code>tail -n 50 -f app.log</code>
grep	Searches text using regular expression patterns	<code>grep -i -r "error" /var/log/</code>
find	Searches directory trees based on file attributes	<code>find . -type f -mtime -2</code>
chmod	Updates Discretionary Access Control file permissions	<code>chmod 755 run.sh</code>
chown	Updates file and directory user and group ownership	<code>chown -R user:group /shared</code>
ps	Displays point-in-time process execution metrics	<code>ps aux grep python</code>

```
| top | Real-time process and system resource monitor | top (press q to quit) |
| kill | Sends control and termination signals by Process ID | kill -9 1234 |
| tar | Archives files and directories | tar -czvf archive.tar.gz ./data |
| df | Reports mounted filesystem space utilization | df -h |
| du | Reports directory and file disk space consumption | du -sch /home/* |
| ssh | Opens an encrypted remote command-line session | ssh -p 22 user@host |
| scp | Transfers files securely over an SSH tunnel | scp data.tar.gz user@host:~/ |
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+
```

16.2 Common Linux Errors and Solutions

1. Permission denied

- Root Cause:** The executing user lacks the required DAC permission bits (Read, Write, or Execute) on the target file or its parent directories.
- Resolution:** Verify current permissions using `ls -l`. Add missing permissions with `chmod` (e.g., `chmod u+x script.sh`) or run the command with `sudo` if administrative privileges are required.

2. Command not found

- Root Cause:** The executable binary is not installed, or its directory is not included in the current `$PATH` environment variable.
- Resolution:** Check tool locations using `which <command>` or `find`. If the binary exists in the current directory, invoke it with an explicit relative path: `./<command>`. Otherwise, add its parent directory to your `$PATH`.

3. No space left on device

- Root Cause:** The storage partition has run out of available data blocks, or all inodes are exhausted.
- Resolution:** Check available disk space with `df -h` and check inode capacity with `df -i`. Free up space by deleting temporary files or purging old log files in `/var/log`.

16.3 Linux Technical Interview Preparation (Q&A)

Q1: What happens under the hood when a user executes a command like `ls` in the shell?

Answer:

- 1.

1. The terminal driver captures the keystroke string and passes it to the shell's input buffer.
2. The shell parses the text into the command name (`ls`) and arguments.
3. The shell checks for aliases and built-in commands; if none match, it searches the directories listed in the `$PATH` environment variable in order.
4. Once located, the shell calls the `fork()` system call to duplicate itself into a child process.
5. Inside the child process, the shell invokes `execve()`, which replaces the child's address space with the `/usr/bin/ls` binary, loads required dynamic libraries via `glibc`, and maps memory structures.
6. The kernel schedules the process. `ls` executes system calls (e.g., `getdents64`) to read directory entries and outputs text to file descriptor 1 (`stdout`).
7. The process exits, and the parent shell cleans up the process table entry and re-displays the command prompt.

Q2: Why is the Sticky Bit set on `/tmp`?

Answer: The `/tmp` directory requires full read, write, and execute permissions (`777`) so any user or application can store temporary scratch files. Without the Sticky Bit, any user could delete or rename files belonging to other users. The Sticky Bit ensures that only the file's owner or the root superuser can delete or rename files in that directory.

Q3: What is the operational difference between a hard link and a symbolic (soft) link?

Answer: A hard link creates an additional directory entry pointing directly to an existing on-disk inode number. It shares the same inode, permissions, and data blocks as the original file, and remains valid even if the original filename is deleted. Hard links cannot span across different filesystem partitions. A symbolic link (symlink) is a distinct, small file with its own inode that contains the text path to target files. Symlinks can span across different partitions, but they break (dangle) if the target file is moved or deleted.

16.4 Engineering Glossary

- **Kernel:** The core privileged program that manages system memory, schedules processes, hosts hardware device drivers, and controls system resources.
- **System Call:** The controlled interface that user-space programs use to request privileged operations from the kernel.
-

- **Inode:** A filesystem data structure storing metadata for a file or directory, including size, permissions, and disk block pointers, but excluding its filename.
- **Process Control Block (PCB):** A kernel data structure that tracks process states, execution registers, memory mappings, and scheduling metrics.
- **Context Switch:** The process where the CPU saves the operational state of a running task and restores another task from its PCB to enable multitasking.
- **Paging:** A virtual memory management scheme that stores and retrieves data from secondary storage in fixed-size blocks called pages.
- **Discretionary Access Control (DAC):** A security model where file owners determine access permissions for user, group, and other access classes.
- **Environment Modules:** Dynamic shell configuration utilities used on HPC clusters to load and unload software versions and compiler paths without collisions.